TALLER EVALUATIVO: PRUEBAS END-TO-END (E2E) CON PLAYWRIGHT & CI/CD (TIEMPO: 2 HORAS)

Ingeniería de Software V

FACULTAD DE INGENIERÍA, DISEÑO Y CIENCIAS APLICADAS INTELIGENTES

DEPARTAMENTO DE COMPUTACIÓN Y SISTEMAS

INFORMACIÓN DE LA ACTIVIDAD — IDENTIFICACIÓN

CÓDIGO:	09776 - TIC	**PERÍODO:**	202620 (Semestre 2026-2)
NRC / GRUPO:	11005 / Grupo 001	**PONDERACIÓN:**	15.0 % (Taller Evaluativo) • Duración: 2 Horas
POLÍTICA IAG:	Nivel 3: Colaboración Asistida	**CALIFICACIÓN:**	_____ / 5.0
ESTUDIANTE 1:	_____	**CÓDIGO:**	_____
ESTUDIANTE 2:	_____	**CÓDIGO:**	_____

1 Contexto del Problema & Aseguramiento de Calidad E2E

1.1 Descripción del Desafío de Ingeniería de Software

La organización **Icesi Cloud Solutions** opera una plataforma de comercio desacoplada compuesta por un Backend REST en **Spring Boot 3 (Java 17)** y un cliente web en **React 18 + Vite + TypeScript**.

Aunque el equipo cuenta con pruebas unitarias en Maven que reportan 100 % de éxito, despliegues recientes a producción generaron fallas críticas de usuario: regresiones en formularios asíncronos, inconsistencias entre navegadores (Chromium, Firefox y WebKit) y congelamiento de la interfaz ante caídas de red.

El objetivo de este taller es estructurar una suite profesional de pruebas automatizadas **End-to-End (E2E)** con **Playwright**, implementando el patrón **Page Object Model (POM)**, localizadores resilientes basados en accesibilidad (**W3C** / **ARIA**), simulación de contingencias (**Network Mocking**) e integración continua en **GitHub Actions** con publicación automatizada de reportes y trazas.

Componente	Tecnología & Stack	Puerto / Endpoint
Backend REST	Spring Boot 3.2, Spring Security (JWT), H2	Puerto `8080` (`/api/v1/orders`)
Frontend SPA	React 18, TypeScript, Vite, TailwindCSS	Puerto `3000` (`http://localhost:3000`)
Suite E2E	Playwright Test Runner v1.42+, TypeScript	Motores: Chromium, Firefox, WebKit
CI/CD Automation	GitHub Actions (`e2e-playwright.yml`)	Headless Execution + HTML Report

2 Procedimiento Experimental & Tareas a Desarrollar

2.1 Fase 1: Configuración de Playwright, Tooling y Multi-Navegador (25 min)

En el directorio raíz del proyecto de pruebas:

a) Inicialice la estructura de pruebas asegurando soporte TypeScript y navegadores oficiales:

```
npm init playwright@latest --yes -- --typescript --browsers=all --github-actions
```

b) Configure el archivo central `playwright.config.ts` cumpliendo con los siguientes parámetros:

- `baseURL: 'http://localhost:3000'` con timeout por prueba de 30 segundos.
- Política de reintentos en CI: `retries: process.env.CI ? 2 : 0` para mitigar latencias de red.
- Diagnóstico: `trace: 'on-first-retry'`, capturas automáticas (`screenshot: 'only-on-failure'`) y grabación (`video: 'retain-on-failure'`).
- Proyectos de ejecución multi-motor: `chromium`, `firefox` y `webkit`.

c) Incorpore en `package.json` los scripts: `"test:e2e": "playwright test"`, `"test:e2e:ui": "playwright test --ui"`, y `"test:e2e:repo` `"playwright show-report"`.

2.2 Fase 2: Patrón Page Object Model (POM) y Flujos Críticos de Negocio (45 min)

Desacople la interacción de la interfaz de la lógica de aserción creando clases modulares en `e2e/pages/`:

a) **Clase** `LoginPage.ts`**:**

- Encapsule la navegación a la vista (`goto('/login')`).
- Defina localizadores semánticos basados en ARIA: `page.getByRole('textbox', { name: /usuario|correo/i })`, `page.getByLabe` y `page.getByRole('button', { name: /iniciar sesión|ingresar/i })`.
- Implemente el método `async login(user, pass)` y el localizador de alertas: `page.getByRole('alert')`.

b) **Clase** `CatalogPage.ts`**:**

- Localizador para el buscador: `page.getByPlaceholder(/buscar productos/i)`.
- Método para filtrar y seleccionar un producto: `page.getByRole('article').filter({ hasText: name })`.
- Método para agregar al carrito y consultar la cantidad del badge: `getCartBadgeCount()`.

c) **Clase** `CheckoutPage.ts`**:**

- Validación de subtotales y diligenciamiento de entrega.
- Confirmación de compra con `page.getByRole('button', { name: /confirmar orden/i })`.
- Verificación del modal de confirmación con el identificador de orden generado.

d) **Suite de Pruebas (**`tests/e2e/order-lifecycle.spec.ts`**):**

- *Test 1 (Camino Feliz):* Login → Catálogo → Agregar al carrito → Checkout → Aserción web-first de compra exitosa (`expect(modal).toBeVisible()`).
- *Test 2 (Credenciales Inválidas):* Intento de acceso erróneo → Aserción de alerta visible sin redirección de URL (`expect(page).toHave`
- *Test 3 (Restricciones):* Comprobar que el botón de pago esté deshabilitado (`toBeDisabled()`) con carrito vacío.

2.3 Fase 3: Interceptación de Red (Network Mocking) y Resiliencia (25 min)

En el archivo `tests/e2e/network-resilience.spec.ts`, valide la tolerancia a fallos del frontend:

a) Simulación de Error 500 en Checkout: Interceptar con `page.route()` la petición `POST **/api/v1/orders` forzando código HTTP 500:

```
await page.route('**/api/v1/orders', async (route) => {
  await route.fulfill({ status: 500, contentType: 'application/json',
    body: JSON.stringify({ message: 'Error interno en pasarela de pagos' }) });
});
```

Valide mediante aserciones web-first que la interfaz muestre el diálogo de reintento sin congelarse.

b) Simulación de Latencia de Red: Intercepte `GET **/api/v1/products` agregando un retraso artificial de 3000 ms. Verifique que durante la espera se despliegue el indicador visual (*spinner* o *skeleton*) y se oculte tras completar la carga.

c) Diagnóstico con Trace Viewer: Ejecute `npx playwright test --trace on` e inspeccione con `npx playwright show-trace` eventos DOM, llamadas HTTP y capturas paso a paso.

2.4 Fase 4: Pipeline Declarativo en GitHub Actions (15 min)

Configure el flujo de integración continua en `.github/workflows/e2e-playwright.yml`:

a) Disparador ante eventos `push` y `pull_request` hacia ramas de entrega.

b) Job sobre `ubuntu-latest` con instalación de Node 20 y navegadores con dependencias:

```
- name: Install dependencies & Playwright Browsers
  run: npm ci && npx playwright install --with-deps
- name: Run Playwright tests
  run: npx playwright test
```

c) Publicación del reporte HTML como artefacto descargable con retención de 30 días (`if: always()`):

```
- uses: actions/upload-artifact@v4
  if: always()
  with: { name: playwright-report, path: playwright-report/ }
```

3 Cuestionario de Análisis Conceptual & Bitácora IAG (10 min)

Responda brevemente a las siguientes preguntas con base en la práctica de laboratorio:

1. **Localizadores Semánticos vs Frágiles:** ¿Por qué se desaconseja XPath o CSS acoplados al DOM en favor de métodos ARIA (`getByRole`)? ¿Qué ventajas otorga ante refactorizaciones y accesibilidad WCAG?
2. **Aislamiento de Contextos (*BrowserContext*):** Compare la apertura de contextos en una sola instancia de Chromium frente a levantar procesos independientes (Selenium) en velocidad, memoria y aislamiento.
3. **Network Mocking vs E2E Puros:** ¿Cuándo conviene interceptar llamadas con `page.route()` y cuándo es indispensable probar contra microservicios y bases de datos reales? ¿Qué riesgos acarrea abusar de mocks?
4. **Pruebas Intermitentes (*Flaky Tests*):** Explique cómo el *auto-waiting* de Playwright elimina pausas fijas (`sleep`). ¿Es la política de `retries` en CI una solución definitiva o paliativa?
5. **Bitácora y Uso Ético de IAG (Nivel 3):** ¿Qué herramientas de IA utilizó para estructurar clases POM o fixtures? ¿Qué alucinaciones o errores detectó y cómo los corrigió técnicamente?

4 Rúbrica de Evaluación Modularizada por Punto (Escala 0.0 a 5.0)

Punto Entregable	Pts	Excelente (4.5 – 5.0)	Aceptable (3.0 – 4.4)	Insuficiente (0.0 – 2.9)
2.1. Configuración Playwright & Tooling	**0.8**	Configuración completa en `playwright.config.ts` multi-navegador (Chromium, Firefox, WebKit), trazas en fallo y scripts npm limpios.	Configuración básica en un solo motor de renderizado o sin políticas de diagnóstico en fallos.	Proyecto no inicializa correctamente o faltan dependencias del sistema operativo.
2.2. Page Object Model & Flujos Críticos	**1.5**	Clases POM limpias en `pages/`, uso exclusivo de localizadores semánticos (`getByRole`), flujo completo de compra y aserciones web-first.	Flujos funcionales pero mezcla selectores directos en los tests o usa selectores CSS frágiles.	Pruebas incompletas, ausencia de POM o uso abusivo de pausas fijas (`waitForTimeout`).
2.3. Network Mocking & Resiliencia	**1.0**	Interceptación elegante con `page.route()` simulando error 500 y latencia, verificación de retroalimentación en UI y uso de Trace Viewer.	Simulación básica de mock sin validación de estados de carga ni manejo de errores en UI.	No implementa interceptación de red o pruebas fallan por mal manejo de promesas.
2.4. CI/CD Pipeline en GitHub Actions	**1.0**	Workflow automatizado, instalación de navegadores con dependencias (`--with-deps`), ejecución headless y reporte HTML con `if: always()`.	Pipeline funcional pero sin persistencia adecuada de artefactos o falla intermitentemente.	Pipeline inexistente, sintaxis YAML inválida o permanentemente en estado fallido (*Failing*).
3.1. Cuestionario & Bitácora IAG	**0.7**	Respuestas técnicas profundas a las 5 preguntas del cuestionario con análisis riguroso y bitácora ética y crítica de uso de IAG.	Respuestas breves con justificación superficial o bitácora de prompts incompleta.	Respuestas erróneas conceptualmente o ausencia de la sección de análisis.
CALIFICACIÓN TOTAL	**5.0**	**Escala Institucional Universidad Icesi: Aprobatoria $\geq$ 3.0 • Duración Máxima: 2 Horas**		

INSTRUCCIONES DE ENTREGA

MODALIDAD

1. **Repositorio en GitHub:** Subir el código fuente completo con `pages/`, `tests/`, configuración de Playwright y workflow en `.github/workflows/e2e-playwright.yml`. La rama evaluada debe exhibir el último commit con el pipeline en **Verde (*Passing*)**.
2. **Documento de Resultados (PDF):** Adjuntar en la plataforma institucional este documento debidamente diligenciado con:
   - Nombres y códigos de los dos integrantes del equipo.
   - Enlace web al repositorio en GitHub.
   - Captura de pantalla del reporte HTML de Playwright (`playwright-report/index.html`) con el 100 % de las pruebas pasando en los tres motores.
   - Captura de pantalla de la ejecución exitosa de la acción en GitHub Actions con el artefacto descargado.
   - Respuestas detalladas al cuestionario conceptual y bitácora ética de uso de IAG de la Sección 3.